



TEMA IV

DESARROLLO DE APLICACIONES WEB EN BACKEND

*"Siempre parece imposible,
hasta que se hace"*
Nelson Mandela

9. CRUD : VISTAS BASADAS EN CLASES

Con Django 1.3 se introducen las "Class based views", una funcionalidad que nos permite implementar nuestras vistas como clases y que además intenta solucionar el no tener que escribir siempre el mismo tipo de código cuando mostramos una página web o hacemos un mantenimiento asociado a un modelo de datos.

Las vistas basadas en clases proporcionan una forma alternativa de implementar vistas como objetos Python en lugar de funciones. No las sustituyen, pero tienen ciertas diferencias y ventajas.

- La orientación a objetos puede utilizarse para factorizar el código y hacerlo **reutilizable**. Podemos extender la clase, sobrescribir métodos...
- En lugar de usar ramificación condicional sobrescribimos métodos

```
from django.http import HttpResponse
def my_view(request):
    if request.method == 'GET':
        return 'result'
class MyView(View):
    def get(self, request):
        return 'result'
```

Al principio solo existían las vistas de funciones. Pasamos a una función una `HttpRequest` y retornamos una `HttpResponse`. Las vistas genéricas basadas en clases se crearon con el mismo objetivo que las vistas genéricas basadas en funciones. Sin embargo, proporcionan un conjunto de herramientas que hace que sean más extensibles y flexibles que las basadas en funciones.

El conjunto de herramientas de clases base que Django usa para construir vistas genéricas basadas en clases están creadas para una máxima flexibilidad y, como tales, tienen muchos métodos y atributos predeterminados que podemos usar o sobrescribir.

Aunque podemos utilizar nuestras propias `class based views`, Django nos da ya hechas las más frecuentes y un conjunto de clases que podemos usar en caso de que ninguna nos encaje. Es en estas clases donde está gran parte de la potencia de las `class based views`. Para usarlas tenemos que entender el concepto de `Mixin`, puesto que eso son estas clases: `Python mixins`.

¿Qué es un Mixin?

Un `Mixin` es una clase que no está concebida para tener entidad por sí misma, sino para extender la funcionalidad otras clases usando la herencia múltiple de Python. Un `Mixin`, por lo tanto, añade funcionalidad a las clases. Podemos crear `Mixins` como si de módulos Python se tratara y aplicarlos a nuestras clases para dotarlas de más funcionalidad.

Recordemos como funciona la herencia múltiple en Python, se ejecutará el primer método que se encuentre yendo de izquierda a derecha en la definición de las clases.

```
class Test(OneMixin, AnotherMixin):  
    pass
```

Python primero buscará en `OneMixin` y si no lo encuentra irá a `AnotherMixin`. El orden en que ponemos las clases es importante. Los `mixins` tienen acceso a la instancia `self` de la clases, y por lo tanto pueden usar información asociada al objeto y otros métodos.

¿Qué es un decorador?

Un decorador es el nombre de un patrón de diseño. Los decoradores alteran de manera dinámica la funcionalidad de una función, método o clase sin tener que hacer subclases o cambiar el código fuente de la clase decorada

Django nos proporciona documentación sobre sus clases basadas en vistas <https://ccbv.co.uk>

Classy Class-Based Views.
Detailed descriptions, with full methods and attributes, for each of Django's class-based generic views.

Did you know?
ccbv.co.uk/ClassName/ will take you straight to the class you're looking for.

Start Here for Django 4.0. [Show more](#)

- AUTH VIEWS**
 - [LoginView](#)
 - [LogoutView](#)
 - [PasswordChangeDoneView](#)
 - [PasswordChangeView](#)
 - [PasswordResetCompleteView](#)
 - [PasswordResetConfirmView](#)
 - [PasswordResetDoneView](#)
 - [PasswordResetView](#)
- GENERIC DETAIL**
 - [DetailView](#)
- GENERIC EDIT**
 - [CreateView](#)
 - [DeleteView](#)
 - [DeleteViewCustomDeleteWarning](#)
 - [FormView](#)
 - [UpdateView](#)
- GENERIC BASE**
 - [RedirectView](#)
 - [TemplateView](#)
 - [View](#)
- GENERIC LIST**
 - [ListView](#)
- GENERIC DATES**
 - [ArchiveIndexView](#)
 - [DateDetailView](#)

What are class-based views anyway?
Django's class-based generic views provide abstract classes implementing common web development tasks. These are very powerful, and heavily-utilise Python's object orientation and multiple inheritance in order to be extensible. This means they're more than just a couple of generic shortcuts — they provide utilities which can be mixed into the much more complex views that you write yourself.

Great! So what's the problem?
All of this power comes at the expense of simplicity. For example, trying to work out exactly which method you need to customise, and what its keyword arguments are, on your `UpdateView` can feel a little like wading through spaghetti — it has 10 separate ancestors (plus `object`), spread across 3 different python files. This site shows you exactly what you need to know.

How does this site help?
To make things easier, we've taken all the attributes and methods that every view defines or inherits, and flattened all that information onto one comprehensive page per view. Check out [UpdateView](#), for example.

Para utilizarlas basta con ver en esta documentación ejemplos.

Empecemos con un ejemplo sencillo de la clase más básica.

Las Clases View y TemplateView

La clase View es el núcleo, si ninguna de las funcionalidades que nos proporciona Django nos sirve, casi seguro que empezaremos a crear nuestra propia clase extendiendo de ésta.

La otra clase básica es TemplateView que hereda de la anterior y nos servirá para el caso más básico, es decir sólo queremos mostrar un template (ésto lo haremos casi siempre)

Ejemplo_1: Supongamos que queremos mostrar una página web, que denominaremos index.html.

Utilizaremos la clase TemplateView, además de View también hereda de TemplateResponseMixin. TemplateResponseMixin define :

1. Dos métodos: `render_to_response` y `get_templates_names`
2. Dos atributos: `template_name` y `response_class`.

Lo que nos interesa en estos momentos es que podemos añadir a nuestra clase la renderización de la plantilla que le pasemos dentro de `template_name` o bien que devolvamos a partir de la sobreescritura del método `get_templates_names`.

Pasos a seguir una vez creados proyecto, app y configurado settings

- a) Creo un template index.html
- b) En el archivo views añado mi vista de clase

```
from django.shortcuts import render
from django.views.generic.base import TemplateView

# Create your views here.

class IndexView(TemplateView):
    template_name='core/index.html'
```

- c) Configuro tanto urls de proyecto como de app

```
from django.contrib import admin
from django.urls import path,include

urlpatterns = [
    path('admin/', admin.site.urls),
    path('',include('core.urls')),
]
```

```
from django.urls import path
from .views import IndexView

urlpatterns = [
    path('',IndexView.as_view(),name='home'),
]
```

El solucionador de URL de Django espera enviar la solicitud y los argumentos asociados a una función no a una clase. Las vistas basadas en clases tienen un método `as_view()` que devuelve una función invocable. Esta función crea una instancia de la clase, llama a `setup()` para inicializar los atributos de la misma y luego llama al método `dispatch()` (de la clase). Este método examina la solicitud para determinar si es GET, POST,... y la pasa al método que coincida. Lo que se retorna es por tanto lo mismo que en una vista basada en funciones, `HttpResponse`.

Ejemplo_2: Vamos a pasar algún parámetro

Al igual que en las vistas basadas en funciones, los parámetros deben pasarse en un contexto(diccionario) por tanto no podemos utilizar sólo el atributo de la clase, debemos sobrescribir el método `get` para poder utilizar `render` con sus correspondientes argumentos como hasta ahora hacíamos

```
from django.shortcuts import render
from django.views.generic.base import TemplateView

# Create your views here.

class IndexView(TemplateView):
    template_name='core/index.html'

    def get(self,request,*args,**kwargs):
        return render(request,self.template_name,{'texto':"Pasando datos"})
```

En el template sólo tenemos que escribir la variable `texto`

Ejemplo_3: Supongamos ahora que alguien intenta hacernos un post sin que nosotros queramos

En las vistas de funciones nuestro código sería algo así

```
if request.method=='POST':
    #ha esto
else:
    #haz esto otro
```

Ahora queda más elegante

```
class IndexView(TemplateView):
    template_name='core/index.html'

    def get(self,request,*args,**kwargs):
        return render(request,self.template_name,{'texto':"Pasando datos"})

    def post(self,request,*args,**kwargs):
        return render(request,'acceso_indebido.html')
```

Como que nos queremos saltar la protección CSRF hemos de decirle a la vista que no la queremos y sobrescribir el método dispatch que se encarga de la parte de “fontanería” necesaria para verificar de qué método HTTP proviene la llamada y qué función debe ejecutar.

```
from django.shortcuts import render
from django.views.decorators.csrf import csrf_exempt
from django.utils.decorators import method_decorator
from django.views.generic.base import TemplateView

# Create your views here.

class IndexView(TemplateView):
    template_name='core/index.html'

    def get(self,request,*args,**kwargs):
        return render(request,self.template_name,{'texto':"Pasando datos"})

    def post(self,request,*args,**kwargs):
        return render(request,'acceso_indebido.html')

    @method_decorator(csrf_exempt)
    def dispatch(self, *args, **kwargs):
        return super(IndexView, self).dispatch(*args, **kwargs)
```

Si probamos con postman la aplicación vemos que efectivamente funciona en ambos casos y retorna lo que debe.

Aquí nos aparecen los decoradores, los veremos más adelante.

De momento ya podemos apreciar que el mundo de las class based views da para mucho. La posibilidad de sobrescribir funciones, cambiar parámetros e ir combinando mixins hasta obtener la funcionalidad que necesitamos nos permite reutilizar mucho código y de manera elegante.

Tenemos vistas para realizar las operaciones más frecuentes en este tipo de aplicaciones.

Uso de Formularios : CRUD

La tramitación del formulario suele constar de 3 pasos:

1. Inicialmente GET(Formulario en blanco o precargado)
2. Con datos ilegales POST(Por lo general, vuelve a mostrar el formulario y el mensaje de error)
3. Con datos legales POST(Procesar datos y redireccionar)

La implementación de estas funciones a menudo conduce a muchos códigos duplicados. Para evitar esto, Django proporciona una serie de vistas generales basadas en clases para el procesamiento de formularios.

Una vista básica basada en la función que maneja formularios puede parecerse a esto:

```
from django.http import HttpResponseRedirect
from django.shortcuts import render
from .forms import MyForm
def myview(request):
    if request.method == "POST":
        form = MyForm(request.POST)
        if form.is_valid():
            return HttpResponseRedirect('/success/') # <procesar datos limpiados del formulario>
        else:
            form = MyForm(initial={'key': 'value'})
            return render(request, 'form_template.html', {'form': form})
```

Una vista similar basada en la clase podría parecer:

```
from django.http import HttpResponseRedirect
from django.shortcuts import render
from django.views import View
from .forms import MyForm
class MyFormView(View):
    form_class = MyForm
    initial = {'key': 'value'}
    template_name = 'form_template.html'

    def get(self, request, *args, **kwargs):
        form = self.form_class(initial=self.initial)
        return render(request, self.template_name, {'form': form})

    def post(self, request, *args, **kwargs):
        form = self.form_class(request.POST)
        if form.is_valid():
            # <procesar datos limpiados del formulario>
            return HttpResponseRedirect('/success/')

        return render(request, self.template_name, {'form': form})
```

Veamos un ejemplo sencillo

Ejemplo_4 : Mostraremos un formulario para dejar nuestra opinión

The screenshot shows a web browser window with the address bar displaying '127.0.0.1:8000/contact/'. The page has a dark header bar with the text 'Inicio Contacto' on the left and 'Acceder' on the right. Below the header, the form is displayed. It starts with the label 'Name:' followed by a text input field containing the placeholder text 'Introduce Nombre'. Below this is the label 'Message:' followed by a larger text area containing the placeholder text 'Danos tu opinión'. At the bottom of the form is a blue button with the text 'Aceptar'.

Lo primero que debemos hacer es añadir el archivo forms.py e implementar nuestro formularios

```
from django import forms

class ContactForm(forms.Form):
    name = forms.CharField(widget=forms.TextInput(attrs={'class':'form-control', 'placeholder':'Introduce Nombre'}))
    message = forms.CharField(widget = forms.Textarea(attrs={'class':'form-control','placeholder':'Danos tu opinión'}))

    def send_email(self):
        pass
```

A continuación implementamos la vista que mostrará el formulario

```
from django.views.generic.base import TemplateView
from .forms import ContactForm
from django.views.generic.edit import FormView

# Create your views here.

class HomeView(TemplateView):
    template_name='core/home.html'

class ContactView(FormView):
    template_name = 'core/contact.html'
    form_class = ContactForm

    def form_valid(self, form):
        #This method is called when valid form data has been POSTED. It should return an HttpResponseRedirect.
        form.send_email()
        return super(ContactView, self).form_valid(form)
```

Añadimos los templates home y contact

Formularios y Modelos

El uso de estas vistas y formularios es realmente interesantes cuándo usamos un modelo, que crearán de forma automática ellas, siempre que sepan qué clase de modelo utilizar.

El formulario modelo proporciona una función form_valid() y guarda automáticamente el modelo. El uso de vistas como CreateView, UpdateView y DeleteView nos facilita enormemente el trabajo porque

todas las operaciones de acceso a la bbdd están implementadas de forma automática. Ya veremos en la práctica que hagamos la cantidad de código que ahorramos.

Cambiamos el ejemplo anterior para que tanto el nombre como el comentario se añadan a una bbdd, necesitamos por tanto un modelo y utilizar las vistas de clase CRUD

Ejemplo_5 : Añadimos un modelo al ejemplo anterior, vamos a dar de alta una opinión

Tendremos que dar los siguientes pasos:

a) Implementar el modelo: con los campos que queremos almacenar en la bbdd

```
from django.db import models

# Create your models here.
class Opinion(models.Model):
    name=models.CharField(verbose_name='Nombre',max_length=100)
    content=models.TextField(verbose_name='Mensaje',max_length=500)
    created=models.DateTimeField(auto_now_add=True, verbose_name='Fecha creación')
    updated=models.DateTimeField(auto_now=True,verbose_name='Fecha de modificación')

    class Meta:
        verbose_name='opinión'
        verbose_name_plural='opiniones'
        ordering =['name']

    def __str__(self):
        return self.name
```

b) Implementar de nuevo forms.py para utilizar este modelo

```
class ContactCreate(CreateView):
    model=Opinion
    form_class = ContactForm
    #Hay que indicar dónde vamos en caso de éxito en la operación
    def get_success_url(self):
        return reverse('home')

    #0 bien
    """success_url=reverse_lazy('home')

    def dispatch(self, request, *args, **kwargs):
        return super(ContactCreate,self).dispatch(request, *args, **kwargs)"""
```

Ahora ya no heredamos de Form sino de ModelForm

c) Implementar de nuevo la vista

```
from django import forms
from .models import Opinion

class ContactForm(forms.ModelForm):

    class Meta:
        model=Opinion
        fields=['name','content']
        widgets={
            'name': forms.TextInput(attrs={'class':'form-control', 'id':'name', 'placeholder':'Nombre', 'required':'required'}),
            'content': forms.Textarea(attrs={'class':'form-control', 'id':'content', 'placeholder':'Comentario', 'required':'required'})
        }

    > static
    v templates/core
    <> base.html
    <> home.html
    <> opinion_form.html
```

Tenemos que heredar de `CreateView`. Esta clase necesita que especifiquemos la url que debe renderizar en caso de que todo vaya bien. Además debemos cambiar el nombre del template porque él buscará un template que se llame **modelo_form.html**

d) Obviamente debemos cambiar las urls porque nuestra vista se llama ahora `ContactCreate`

En la práctica de este tema tendrás ocasión de utilizar las otras vistas CRUD: Delete, Update

Vista ListView

`ListView` es un componente ideal para hacer búsquedas y filtros. Con una hoja de estilo y un poco de javascript podemos hacer prácticamente de todo.

Un ejemplo sencillo, queremos establecer dos filtros: uno para cantidades positivas y uno para cantidades negativas, además de la opción por defecto que será la de mostrar todo. En la clase lo que haremos será extender el método `get_queryset` de manera que tenga en cuenta si estamos filtrando o no y que aplique el filtro que corresponda en cada caso:

```
class SampleListView(ListView):
    model = Sample
```

```
paginate_by = 5

def get_queryset(self):
    queryset = super(SampleListView, self).get_queryset()
    filter = self.request.GET.get('filter')
    if filter == 'positive':
        queryset = queryset.filter(ammount__gt=0)
    elif filter == 'negative':
        queryset = queryset.filter(ammount__lt=0)
    return queryset
```

Como podemos ver en los ejemplo, al final sólo se trata de ir buscando lo que se debe sobrescribir o extender para adaptarlo a nuestra necesidades.